

EMA Implementation - Complete Logic & Code

EMA Formula Breakdown

EMA = Exponential Moving Average

EMA gives more weight to recent prices, making it respond faster to price changes than SMA.

Formula:

$$\text{EMA} = (\text{Current Price} \times K) + (\text{Previous EMA} \times (1 - K))$$

Where:

$$K (\text{Multiplier}) = 2 / (\text{Period} + 1)$$

Example with Period 20:

$$K = 2 / (20 + 1) = 2 / 21 = 0.0952$$

So recent price = 9.52% weight

Previous EMA = 90.48% weight

EMA Calculator Class

javascript

```
// emaCalculator.js

class EMACalculator {
  constructor(length = 200, source = 'close', smoothingLine = 'SMA', smoothingLength = 1, offset = 0) {
    this.length = length;
    this.source = source; // 'close', 'open', 'high', 'low', 'hl2', 'hlc3', 'ohl4'
    this.smoothingLine = smoothingLine; // 'SMA', 'EMA', 'WMA'
    this.smoothingLength = smoothingLength;
    this.offset = offset;
  }

  // MAIN: Calculate EMA
  calculate(candles) {
    if (candles.length < this.length + 1) {
      return new Array(candles.length).fill(undefined);
    }

    // Step 1: Extract source values
    const sourceValues = this.extractSource(candles);

    // Step 2: Calculate base EMA
    const baseEMA = this.calculateBaseEMA(sourceValues);

    // Step 3: Apply smoothing if needed
    let finaleMA = baseEMA;
    if (this.smoothingLine !== 'SMA' || this.smoothingLength > 1) {
      finaleMA = this.smoothEMA(baseEMA, this.smoothingLength, this.smoothingLine);
    }

    // Step 4: Apply offset
    const offsetEMA = this.applyOffset(finaleMA);

    return offsetEMA;
  }

  // Extract source from candle (close, open, high, low, hl2, hlc3, ohlc4)
  extractSource(candles) {
    return candles.map(candle => {
      switch (this.source) {
        case 'open':
          return candle.open;
        case 'high':
          return candle.high;
        case 'low':
          return candle.low;
        case 'close':
          return candle.close;
        case 'hl2':
          return (candle.high + candle.low) / 2;
        case 'hlc3':
          return (candle.high + candle.low + candle.close) / 3;
        case 'ohl4':
          return (candle.open + candle.high + candle.low + candle.close) / 4;
      }
    });
  }
}
```

```

        return candle.low;
    case 'hl2':
        return (candle.high + candle.low) / 2;
    case 'hlc3':
        return (candle.high + candle.low + candle.close) / 3;
    case 'ohlc4':
        return (candle.open + candle.high + candle.low + candle.close) / 4;
    case 'close':
    default:
        return candle.close;
    }
});
}

// Calculate basic EMA with given period
calculateBaseEMA(values) {
    if (values.length < this.length) {
        return new Array(values.length).fill(undefined);
    }

    const ema = new Array(values.length);
    const multiplier = 2 / (this.length + 1);

    // Fill undefined for first length-1 values
    for (let i = 0; i < this.length - 1; i++) {
        ema[i] = undefined;
    }

    // Calculate first EMA (using SMA for initialization)
    let sum = 0;
    for (let i = 0; i < this.length; i++) {
        sum += values[i];
    }
    let emaValue = sum / this.length;
    ema[this.length - 1] = emaValue;

    // Calculate subsequent EMAs
    for (let i = this.length; i < values.length; i++) {
        emaValue = values[i] * multiplier + emaValue * (1 - multiplier);
        ema[i] = emaValue;
    }

    return ema;
}

```

```

// Apply smoothing to EMA values (for smoothing parameter)
smoothEMA(emaValues, smoothingLength, smoothingType) {
  const validValues = emaValues.filter(v => v !== undefined);

  if (smoothingType === 'SMA') {
    return this.smoothWithSMA(emaValues, smoothingLength);
  } else if (smoothingType === 'EMA') {
    return this.smoothWithEMA(emaValues, smoothingLength);
  } else if (smoothingType === 'WMA') {
    return this.smoothWithWMA(emaValues, smoothingLength);
  }

  return emaValues;
}

// Smooth with SMA
smoothWithSMA(values, period) {
  const result = [...values];

  for (let i = period - 1; i < values.length; i++) {
    if (values[i] === undefined) continue;

    let sum = 0;
    let count = 0;

    for (let j = i - period + 1; j <= i; j++) {
      if (values[j] !== undefined) {
        sum += values[j];
        count++;
      }
    }

    if (count === period) {
      result[i] = sum / period;
    }
  }

  return result;
}

// Smooth with EMA
smoothWithEMA(values, period) {
  const result = new Array(values.length);
  const multiplier = 2 / (period + 1);
  let ema = undefined;

```

```

for (let i = 0; i < values.length; i++) {
  if (values[i] === undefined) {
    result[i] = undefined;
    continue;
  }

  if (ema === undefined) {
    ema = values[i];
  } else {
    ema = values[i] * multiplier + ema * (1 - multiplier);
  }

  result[i] = ema;
}

return result;
}

// Smooth with WMA
smoothWithWMA(values, period) {
  const result = new Array(values.length);
  const weights = Array.from({ length: period }, (_, i) => i + 1);
  const weightSum = (period * (period + 1)) / 2;

  for (let i = period - 1; i < values.length; i++) {
    if (values[i] === undefined) continue;

    let weightedSum = 0;
    let count = 0;

    for (let j = 0; j < period; j++) {
      const idx = i - period + 1 + j;
      if (values[idx] !== undefined) {
        weightedSum += values[idx] * weights[j];
        count++;
      }
    }

    if (count === period) {
      result[i] = weightedSum / weightSum;
    }
  }

  return result;
}

```

```
}

// Apply offset (shift the line forward/backward)
applyOffset(values) {
  if (this.offset === 0) return values;

  const result = new Array(values.length);

  if (this.offset > 0) {
    // Shift right
    for (let i = 0; i < values.length; i++) {
      result[i] = i - this.offset >= 0 ? values[i - this.offset] : undefined;
    }
  } else {
    // Shift left
    for (let i = 0; i < values.length; i++) {
      result[i] = i - this.offset < values.length ? values[i - this.offset] : undefined;
    }
  }

  return result;
}
}

export default EMACalculator;
```

EMA Settings Modal Component

javascript

```

import React, { useState } from 'react';
import './EMASettingsModal.css';

const EMASettingsModal = ({ isOpen, onClose, onApply, defaultSettings }) => {
  const [activeTab, setActiveTab] = useState('inputs');
  const [settings, setSettings] = useState({
    length: defaultSettings?.length || 200,
    source: defaultSettings?.source || 'close',
    offset: defaultSettings?.offset || 0,
    smoothingLine: defaultSettings?.smoothingLine || 'SMA',
    smoothingLength: defaultSettings?.smoothingLength || 9,
    plot: defaultSettings?.plot !== false,
    color: defaultSettings?.color || '#2196F3',
    lineWidth: defaultSettings?.lineWidth || '2px',
    lineStyle: defaultSettings?.lineStyle || 'Solid',
    precision: defaultSettings?.precision || 2,
    labelsOnScale: defaultSettings?.labelsOnScale !== false,
    valuesInStatusLine: defaultSettings?.valuesInStatusLine !== false,
  });

  if (!isOpen) return null;

  const handleApply = () => {
    onApply(settings);
    onClose();
  };

  return (
    <div className="ema-modal-overlay">
      <div className="ema-modal">
        /* Header */
        <div className="ema-modal-header">
          <h2>EMA</h2>
          <button className="ema-close-btn" onClick={onClose}>x</button>
        </div>

        /* Tabs */
        <div className="ema-tabs">
          <button
            className={`ema-tab ${activeTab === 'inputs' ? 'active' : ''}`}
            onClick={() => setActiveTab('inputs')}
          >
            Inputs
          </button>

```

```

<button
  className={`ema-tab ${activeTab === 'style' ? 'active' : ''}`}
  onClick={() => setActiveTab('style')}
>
  Style
</button>
</div>

{/* TAB 1: INPUTS */}
{activeTab === 'inputs' && (
  <div className="ema-tab-content">
    <div className="ema-setting">
      <label>Length</label>
      <input
        type="number"
        value={settings.length}
        onChange={(e) => setSettings({ ...settings, length: parseInt(
          min="1"
          max="500"
        />
      </div>

    <div className="ema-setting">
      <label>Source</label>
      <select
        value={settings.source}
        onChange={(e) => setSettings({ ...settings, source: e.target.
      >
        <option value="close">close</option>
        <option value="open">open</option>
        <option value="high">high</option>
        <option value="low">low</option>
        <option value="hl2">hl2</option>
        <option value="hlc3">hlc3</option>
        <option value="ohlc4">ohlc4</option>
      </select>
    </div>

    <div className="ema-setting">
      <label>Offset</label>
      <input
        type="number"
        value={settings.offset}
        onChange={(e) => setSettings({ ...settings, offset: parseInt(
          min="-100"

```



```

        max="100"
      />
    </div>

    <div className="ema-setting">
      <label>Smoothing Line</label>
      <select
        value={settings.smoothingLine}
        onChange={(e) => setSettings({ ...settings, smoothingLine: e.
          >
            <option value="SMA">SMA</option>
            <option value="EMA">EMA</option>
            <option value="WMA">WMA</option>
          </select>
    </div>

    <div className="ema-setting">
      <label>Smoothing Length</label>
      <input
        type="number"
        value={settings.smoothingLength}
        onChange={(e) => setSettings({ ...settings, smoothingLength:
          min="1"
          max="100"
        />
      </div>
    </div>
  )}

  { /* TAB 2: STYLE */ }
  {activeTab === 'style' && (
    <div className="ema-tab-content">
      <div className="ema-plot-section">
        <div className="ema-checkbox">
          <input
            type="checkbox"
            id="plot"
            checked={settings.plot}
            onChange={(e) => setSettings({ ...settings, plot: e.target.
          />
          <label htmlFor="plot">Plot</label>
        </div>

        <div className="ema-style-controls">
          <div className="ema-color-picker">

```

```

        <input
          type="color"
          value={settings.color}
          onChange={(e) => setSettings({ ...settings, color: e.target.value })}
        />
      </div>

      <select
        value={settings.lineWidth}
        onChange={(e) => setSettings({ ...settings, lineWidth: e.target.value })}
        className="ema-line-width"
      >
        <option value="1px">1px</option>
        <option value="2px">2px</option>
        <option value="3px">3px</option>
        <option value="4px">4px</option>
      </select>

      <select
        value={settings.lineStyle}
        onChange={(e) => setSettings({ ...settings, lineStyle: e.target.value })}
        className="ema-line-style"
      >
        <option value="Solid">Solid</option>
        <option value="Dashed">Dashed</option>
        <option value="Dotted">Dotted</option>
      </select>
    </div>
  </div>

  <div className="ema-outputs-section">
    <h4>OUTPUTS</h4>

    <div className="ema-setting">
      <label>Precision</label>
      <select
        value={settings.precision}
        onChange={(e) => setSettings({ ...settings, precision: e.target.value })}
      >
        <option value="0">0</option>
        <option value="1">1</option>
        <option value="2">2</option>
        <option value="3">3</option>
        <option value="4">4</option>
        <option value="5">5</option>
      </select>
    </div>
  </div>

```

```

        </select>
      </div>

      <div className="ema-checkbox">
        <input
          type="checkbox"
          id="labelsOnScale"
          checked={settings.labelsOnScale}
          onChange={(e) => setSettings({ ...settings, labelsOnScale:
        />
        <label htmlFor="labelsOnScale">Labels on price scale</label>
      </div>

      <div className="ema-checkbox">
        <input
          type="checkbox"
          id="valuesInStatusLine"
          checked={settings.valuesInStatusLine}
          onChange={(e) => setSettings({ ...settings, valuesInStatusLine:
        />
        <label htmlFor="valuesInStatusLine">Values in status line</label>
      </div>
    </div>
  </div>
  )}

  {/* Buttons */}
  <div className="ema-modal-footer">
    <button className="ema-btn-cancel" onClick={onClose}>Cancel</button>
    <button className="ema-btn-ok" onClick={handleApply}>Ok</button>
  </div>
</div>
</div>
);
};

export default EMASettingsModal;

```

CSS for EMA Settings Modal

CSS

```
/* EMASettingsModal.css */

.ema-modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.ema-modal {
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
  max-width: 500px;
  width: 90%;
  max-height: 80vh;
  overflow-y: auto;
}

.ema-modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  border-bottom: 1px solid #e8e8e8;
}

.ema-modal-header h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
  color: #1a1a1a;
}

.ema-close-btn {
  background: none;
  border: none;
  font-size: 24px;
```

```
    color: #666;
    cursor: pointer;
    transition: color 0.2s;
}

.ema-close-btn:hover {
    color: #000;
}

/* TABS */
.ema-tabs {
    display: flex;
    border-bottom: 1px solid #e8e8e8;
    background: #fafafa;
}

.ema-tab {
    flex: 1;
    padding: 12px;
    background: none;
    border: none;
    cursor: pointer;
    font-size: 14px;
    font-weight: 500;
    color: #666;
    border-bottom: 2px solid transparent;
    transition: all 0.2s;
}

.ema-tab:hover {
    color: #1a1a1a;
}

.ema-tab.active {
    color: #2196F3;
    border-bottom-color: #2196F3;
    background: #ffffff;
}

/* TAB CONTENT */
.ema-tab-content {
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 16px;
```

```
}

.ema-setting {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.ema-setting label {
  font-size: 13px;
  font-weight: 500;
  color: #1a1a1a;
}

.ema-setting input,
.ema-setting select {
  padding: 10px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 13px;
  background: #ffffff;
  color: #1a1a1a;
}

.ema-setting input:focus,
.ema-setting select:focus {
  outline: none;
  border-color: #2196F3;
  box-shadow: 0 0 0 2px rgba(33, 150, 243, 0.1);
}

/* PLOT SECTION */
.ema-plot-section {
  display: flex;
  flex-direction: column;
  gap: 12px;
  padding: 12px;
  background: #fafafa;
  border-radius: 4px;
}

.ema-checkbox {
  display: flex;
  align-items: center;
  gap: 8px;
```

```
}

.ema-checkbox input[type="checkbox"] {
  width: 18px;
  height: 18px;
  cursor: pointer;
  accent-color: #2196F3;
}

.ema-checkbox label {
  flex: 1;
  cursor: pointer;
  font-size: 14px;
  color: #1a1a1a;
  margin: 0;
  font-weight: 500;
}

.ema-style-controls {
  display: flex;
  gap: 8px;
  align-items: center;
}

.ema-color-picker {
  display: flex;
  align-items: center;
  gap: 4px;
}

.ema-color-picker input[type="color"] {
  width: 40px;
  height: 40px;
  border: 1px solid #ddd;
  border-radius: 4px;
  cursor: pointer;
}

.ema-line-width,
.ema-line-style {
  flex: 1;
  padding: 8px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 13px;
}
```

```
    cursor: pointer;
}

/* OUTPUTS SECTION */
.ema-outputs-section {
    display: flex;
    flex-direction: column;
    gap: 12px;
    padding: 12px;
    background: #fafafa;
    border-radius: 4px;
}

.ema-outputs-section h4 {
    margin: 0;
    font-size: 11px;
    font-weight: 600;
    color: #999;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

/* BUTTONS */
.ema-modal-footer {
    display: flex;
    gap: 8px;
    padding: 16px 20px;
    border-top: 1px solid #e8e8e8;
    background: #fafafa;
}

.ema-btn-cancel,
.ema-btn-ok {
    flex: 1;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.2s;
}

.ema-btn-cancel {
    background: #ffffff;
```



```
    color: #2196F3;
    border-color: #2196F3;
}

.ema-btn-cancel:hover {
    background: #f0f7ff;
}

.ema-btn-ok {
    background: #2196F3;
    color: #ffffff;
    border-color: #2196F3;
}

.ema-btn-ok:hover {
    background: #1976D2;
}
```

Chart Integration with EMA

javascript

```
import React, { useEffect, useRef, useState } from 'react';
import { createChart } from 'lightweight-charts';
import EMACalculator from './emaCalculator';
import EMASettingsModal from './EMASettingsModal';

const TradeZillaChart = ({ data, symbol }) => {
  const chartContainer = useRef(null);
  const [showEMAModal, setShowEMAModal] = useState(false);
  const [emaSettings, setEmaSettings] = useState({
    length: 200,
    source: 'close',
    offset: 0,
    smoothingLine: 'SMA',
    smoothingLength: 9,
    plot: true,
    color: '#2196F3',
    lineWidth: '2px',
    lineStyle: 'Solid',
    precision: 2,
    labelsOnScale: true,
    valuesInStatusLine: true,
  });
  const [emaActive, setEmaActive] = useState(false);
  const chartRef = useRef(null);
  const emaSeries = useRef(null);

  // Initialize chart
  useEffect(() => {
    if (!chartContainer.current) return;

    const chart = createChart(chartContainer.current, {
      layout: {
        textColor: '#1a1a1a',
        backgroundColor: 'ffffff',
      },
      height: 500,
      width: chartContainer.current.clientWidth,
    });

    chartRef.current = chart;

    // Add candlestick series
    const candlestickSeries = chart.addCandlestickSeries();
    const formattedData = data.map(c => ({
```

```

        time: Math.floor(new Date(c.timestamp).getTime() / 1000),
        open: c.open,
        high: c.high,
        low: c.low,
        close: c.close,
    }));

candlestickSeries.setData(formattedData);
chart.timeScale().fitContent();

return () => {
    chart.remove();
};
}, [data]);

// Add EMA indicator
const addEMA = (settings) => {
    if (!chartRef.current) return;

    const chart = chartRef.current;

    // Calculate EMA
    const emaCalc = new EMACalculator(
        settings.length,
        settings.source,
        settings.smoothingLine,
        settings.smoothingLength,
        settings.offset
    );
    const emaValues = emaCalc.calculate(data);

    // Format for chart
    const emaData = data.map((d, i) => ({
        time: Math.floor(new Date(d.timestamp).getTime() / 1000),
        value: emaValues[i],
    })).filter(d => d.value !== undefined);

    // Determine line width
    const lineWidthMap = {
        '1px': 1,
        '2px': 2,
        '3px': 3,
        '4px': 4,
    };
};

```

```

// Determine line style (lightweight-charts uses 0=solid, 1=dashed, 2=dotted)
const lineStyleMap = {
  'Solid': 0,
  'Dashed': 1,
  'Dotted': 2,
};

// Remove previous EMA series if exists
if (emaSeries.current) {
  chartRef.current.removeSeries(emaSeries.current);
}

// Add EMA series
const series = chartRef.current.addLineSeries({
  color: settings.color,
  lineWidth: lineWidthMap[settings.lineWidth] || 2,
  lineStyle: lineStyleMap[settings.lineStyle] || 0,
  title: 'EMA',
});

series.setData(emaData);
emaSeries.current = series;

// Format data for status line display
if (settings.valuesInStatusLine) {
  series.applyOptions({
    customValue: (d) => {
      if (!d.value) return '';
      return `EMA: ${d.value.toFixed(settings.precision)}`;
    },
  });
}

// Format data for price scale labels
if (settings.labelsOnScale) {
  const lastEMA = emaValues.find(v => v !== undefined && v !== null);
  if (lastEMA) {
    series.applyOptions({
      title: `EMA(${settings.length})`,
    });
  }
}

setEmaActive(true);
};

```

```

const handleEMASettings = (newSettings) => {
  setEmaSettings(newSettings);
  addEMA(newSettings);
};

return (
  <div>
    {/* Toolbar */}
    <div style={{ marginBottom: '12px', padding: '8px 12px', background: '#
      <button
        onClick={() => setShowEMAModal(true)}
        style={{
          padding: '8px 16px',
          background: emaActive ? '#2196F3' : '#fff',
          color: emaActive ? '#fff' : '#1a1a1a',
          border: '1px solid #ddd',
          borderRadius: '4px',
          cursor: 'pointer',
          fontSize: '13px',
          fontWeight: '500',
          transition: 'all 0.2s',
        }}
      >
        {emaActive ? '✓ EMA' : 'EMA'}
      </button>
    </div>

    {/* Chart */}
    <div ref={chartContainer} style={{ borderRadius: '8px', overflow: 'hidd

    {/* EMA Settings Modal */}
    <EMASettingsModal
      isOpen={showEMAModal}
      onClose={() => setShowEMAModal(false)}
      onApply={handleEMASettings}
      defaultSettings={emaSettings}
    />
  </div>
);
};

export default TradeZillaChart;

```

Test EMA Calculation

javascript

```
// Test data
const testCandles = [
  { timestamp: '2026-07-01', open: 100, high: 105, low: 99, close: 102 },
  { timestamp: '2026-07-02', open: 102, high: 108, low: 101, close: 106 },
  { timestamp: '2026-07-03', open: 106, high: 110, low: 104, close: 108 },
  { timestamp: '2026-07-04', open: 108, high: 112, low: 107, close: 111 },
  { timestamp: '2026-07-05', open: 111, high: 115, low: 110, close: 113 },
  // ... more candles (need at least 200 for EMA 200)
];

// Calculate EMA
const emaCalc = new EMACalculator(200, 'close', 'SMA', 9, 0);
const emaValues = emaCalc.calculate(testCandles);

console.log('EMA Values:', emaValues);
// Output: [undefined, undefined, ..., 102.5, 104.2, 106.1, 108.3, 110.5]
```

✅ Features Implemented

✅ **Length** - Adjustable (1-500) ✅ **Source** - close, open, high, low, hl2, hlc3, ohlc4 ✅
Offset - Shift line forward/backward (-100 to 100) ✅ **Smoothing Line** - SMA, EMA, WMA ✅ **Smoothing Length** - Adjustable (1-100) ✅ **Color picker** - Choose any color ✅
Line width - 1px, 2px, 3px, 4px ✅ **Line style** - Solid, Dashed, Dotted ✅ **Precision** -
Decimal places (0-5) ✅ **Labels on price scale** - Show EMA value on right ✅ **Values in status line** - Show value when hovering ✅ **Settings modal** - Inputs & Style tabs

Quick Start

1. Copy EMA Calculator

javascript

```
// Save as emaCalculator.js
class EMACalculator { ... }
```

2. Copy Settings Modal

```
javascript

// Save as EMASettingsModal.jsx
const EMASettingsModal = ({ ... }) => { ... }
```

3. Copy CSS

```
css

/* Save as EMASettingsModal.css */
```

4. Integrate into Chart

```
javascript

// Use in your chart component
import EMACalculator from './emaCalculator';
import EMASettingsModal from './EMASettingsModal';
```

That's it! Full EMA with all your settings. 🚀